

Proyecto # 1 - CEDigital

Documentación Técnica

José Bernardo Barquero Bonilla

2023150476

Jimmy Feng Feng

2023060347

Alexander Montero Vargas

2023166058

Adrián Muñoz Alvarado

2023207992

Diego Salas Ovares

2023107645

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: CE3101 - Bases de Datos

Profesor: Marco Rivera Meneses

22 de Mayo de 2025

1 Descripción de los Algoritmos y Métodos Implementados

En esta sección se describen en detalle los principales algoritmos y métodos implementados en el sistema, tanto en el backend como en el frontend.

1.1 Backend

1.1.1 Implementación de Métodos CRUD para SQL Server

En el backend se implementaron múltiples controladores en C# para manejar las operaciones sobre entidades almacenadas en la base de datos relacional en SQL Server. Estos controladores siguen el estilo RESTful y utilizan Entity Framework Core como ORM para facilitar la persistencia de datos. Cada controlador expone un conjunto de métodos que corresponden a operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre una entidad del dominio del sistema.

A continuación, se describe de forma general el comportamiento de estos métodos:

- **GET:** Se implementaron métodos que permiten obtener uno o varios registros de cada entidad. Por ejemplo, obtener una carrera por su identificador, o listar todos los cursos. Estas consultas se realizan de manera asíncrona utilizando métodos como `FindAsync` y `ToListAsync`.
- **POST:** Permite crear nuevos registros en la base de datos. Antes de insertar los datos, se realiza una validación básica para asegurarse de que el objeto recibido no sea `null`, y en algunos casos, se validan campos específicos como el formato del período en los semestres.
- **PUT:** Se utilizan para modificar registros existentes. Se valida que el identificador en la URL coincida con el del cuerpo de la solicitud, y se actualizan los campos correspondientes utilizando el método `Entry().State = EntityState.Modified`.
- **DELETE:** Se permite eliminar registros a partir de su identificador. Antes de eliminar, se valida que el recurso exista. En caso contrario, se responde con un código HTTP 404.

Cada método devuelve respuestas HTTP adecuadas como 200 OK, 201 Created, 204 No Content, 400 Bad Request o 404 Not Found, facilitando así el consumo desde el frontend. Además, se hace uso de `CreatedAtAction` para retornar la ubicación del nuevo recurso tras una operación POST, siguiendo las mejores prácticas REST.

Estos controladores conforman la base lógica del backend del sistema, y permiten la gestión estructurada de elementos clave como carreras, cursos, semestres y grupos académicos.

En la **Figura 1** se puede apreciar un diagrama de flujo para estos métodos CRUD implementados.

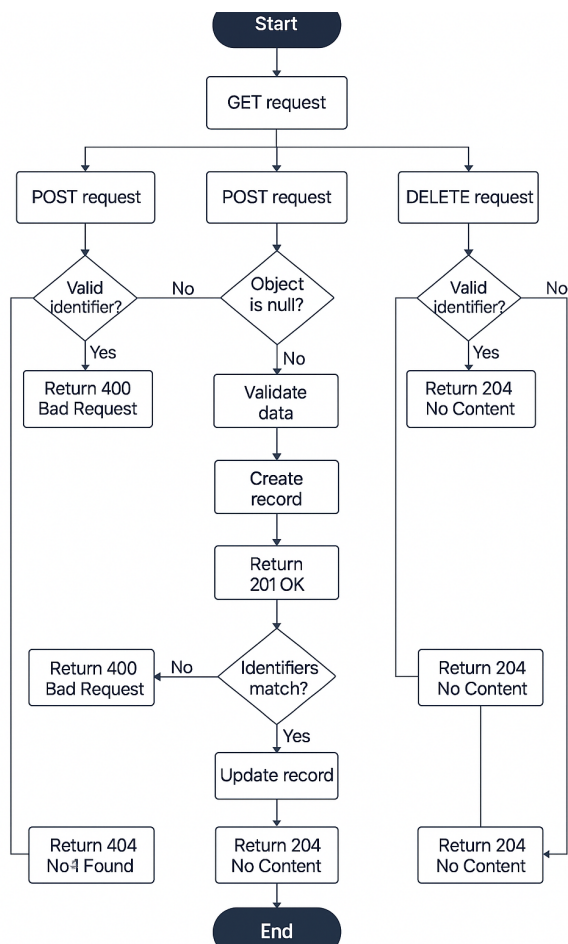


Figure 1: Diagrama de flujo para los métodos CRUD del SQL

1.1.2 Algoritmo de Autenticación mediante MongoDB

La autenticación de los usuarios en el sistema se realiza a través de un controlador especializado llamado **AuthController**. Este controlador forma parte del conjunto de controladores vinculados a la base de datos no relacional MongoDB, ya que toda la información personal de profesores, estudiantes y administradores está almacenada en dicha base, de acuerdo con los requerimientos del sistema.

El método implementado es un **POST** hacia la ruta **/api/auth/login**. Este recibe un objeto JSON con dos atributos: **correo** y **password**. Internamente, se utiliza un servicio denominado **AuthService** para validar las credenciales proporcionadas. Este servicio realiza

una consulta a MongoDB y compara la contraseña encriptada con la almacenada en la base de datos.

El flujo de ejecución puede describirse de la siguiente forma:

- Se valida que el objeto de entrada contenga los datos requeridos.
- Se llama a la función `LoginAsync(correo, password)` del `AuthService`.
- El servicio verifica si existe un usuario con el correo especificado y compara la contraseña utilizando cifrado MD5 como mínimo.
- En caso de que las credenciales no sean válidas, se responde con `401 Unauthorized` y un mensaje de error.
- Si las credenciales son válidas, se responde con `200 OK` y se retorna el rol del usuario (estudiante, profesor o administrador).

Este método permite establecer la sesión de usuario de forma centralizada y basada en roles. La autenticación es fundamental para redirigir al usuario a la vista correspondiente del sistema y controlar el acceso a los recursos protegidos de cada API.

En la **Figura 2** se puede apreciar un diagrama de flujo para este método de autenticación.

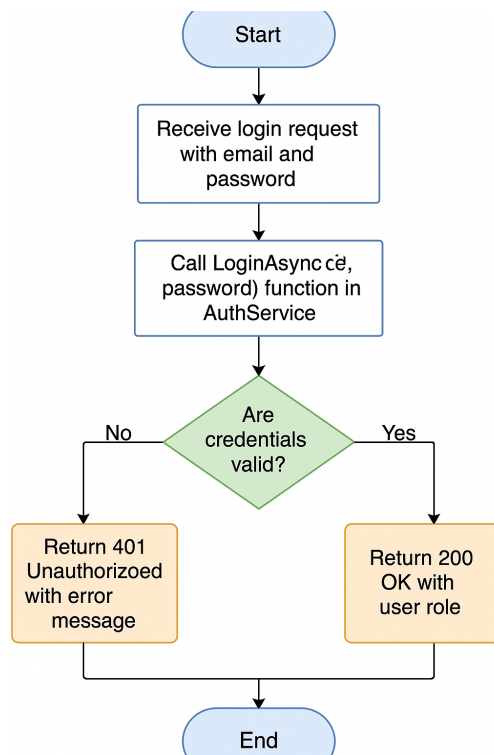


Figure 2: Diagrama de flujo para el método de autenticación

1.1.3 Implementación de Métodos CRUD para MongoDB

Como parte del cumplimiento de los requerimientos del sistema, toda la información personal de estudiantes y profesores se gestiona mediante una base de datos no relacional en MongoDB. Para este fin, se desarrollaron dos controladores principales: `EstudianteController` y `ProfesorController`, los cuales se comunican con sus respectivos servicios (`EstudianteService` y `ProfesorService`) para realizar las operaciones.

A diferencia del acceso a datos en SQL Server, aquí se utiliza un enfoque basado en colecciones y documentos, y las operaciones CRUD se realizan de manera asíncrona utilizando el driver oficial de MongoDB para .NET.

- **GET:** Ambos controladores cuentan con un método `GetAll` que permite recuperar la lista completa de estudiantes o profesores desde sus respectivas colecciones. Esta información es esencial para poblar vistas administrativas y validar identidades en otras partes del sistema.
- **POST:** Se implementó un método para registrar un nuevo estudiante o profesor en la base de datos. Los documentos son insertados directamente en MongoDB utilizando el método `CreateAsync` del servicio correspondiente. Cada documento incluye campos como cédula, nombre, correo, teléfono y contraseña cifrada.
- **DELETE:** Permite eliminar un registro mediante el identificador único (`carnet` para estudiantes y `cédula` para profesores). Antes de eliminar, se realiza una validación para asegurar que el documento existe. Si no se encuentra, se retorna un código HTTP 404 Not Found.

Estas operaciones permiten la integración eficiente de MongoDB con el resto del sistema, ofreciendo una forma desacoplada de manejar información sensible que no debe almacenarse en la base de datos relacional. Todos los métodos siguen el estilo REST y retornan códigos de estado HTTP adecuados como 200 OK, 201 Created, 204 No Content y 404 Not Found.

En la **Figura 3** se puede apreciar un diagrama de flujo para los métodos CRUD de Mongo DB para el estudiante y profesor.

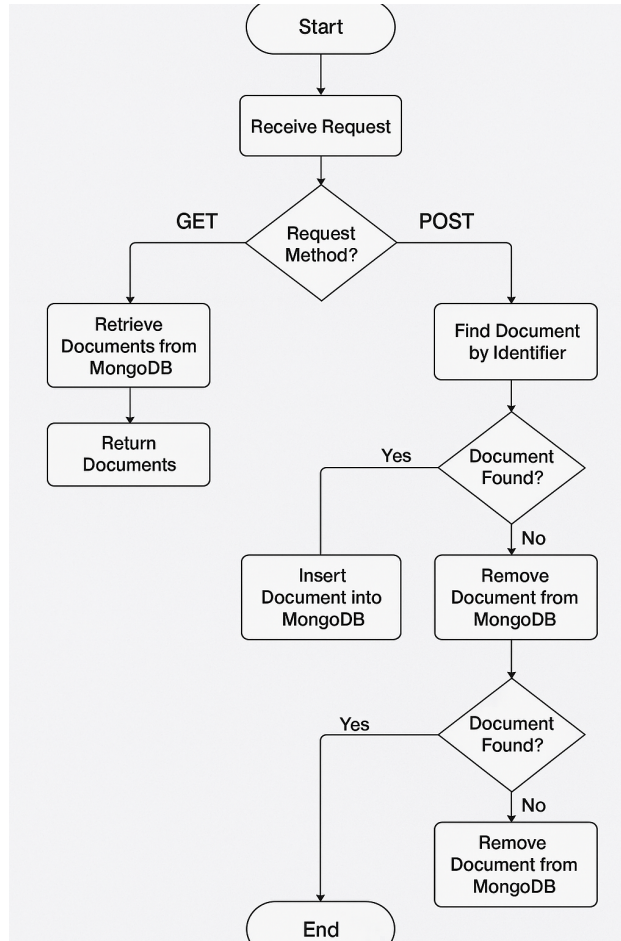


Figure 3: Diagrama de flujo para los métodos CRUD de MongoDB del estudiante y profesor

1.1.4 Inicialización Masiva de Semestres, Grupos y Personas desde Excel

Este algoritmo permite la carga masiva de información académica a partir de un archivo Excel estructurado. El proceso inicia con la lectura del archivo, identificando filas relevantes para semestres, grupos y asignaciones de personas a grupos. Para cada fila, se extraen los datos de año, periodo, estado, curso, número de grupo y datos personales. El sistema valida la unicidad de cada semestre y grupo antes de insertarlos en la base de datos, evitando duplicados. Además, se crean carpetas y rubros por defecto para cada grupo nuevo. Finalmente, se asignan estudiantes y profesores a los grupos correspondientes, validando la existencia previa de cada persona y evitando asignaciones duplicadas. El algoritmo reporta errores detallados si encuentra inconsistencias, como periodos inválidos, estados incorrectos, cursos inexistentes o personas no registradas, y asegura la integridad de la información cargada.

En la **Figura 4** se puede apreciar el diagrama de flujo para este algoritmo.

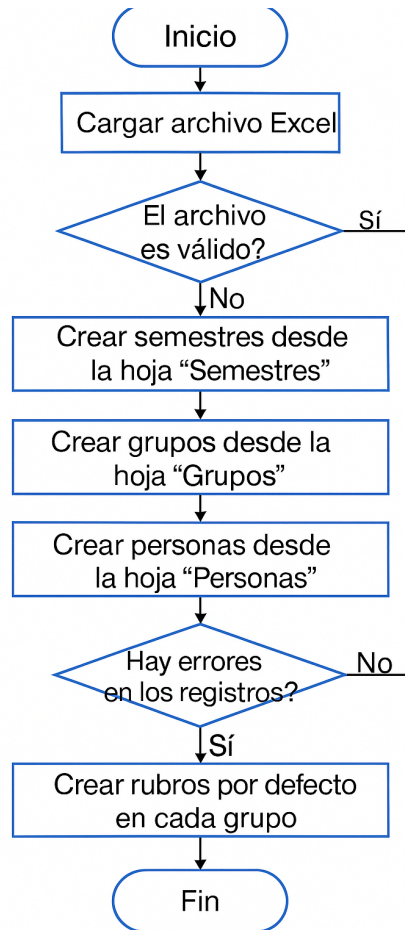


Figure 4: Diagrama de flujo para la inicialización masiva usando un archivo excel

1.1.5 Cálculo de Notas por Grupo

Este método está diseñado para calcular y estructurar las calificaciones de todos los estudiantes de un grupo en un curso específico. Utiliza una consulta SQL avanzada que recupera, para cada estudiante, las calificaciones obtenidas en cada evaluación, agrupadas por rubro. Posteriormente, el algoritmo agrupa los resultados por estudiante y, dentro de cada estudiante, por rubro de evaluación. Para cada rubro, calcula el promedio de las calificaciones obtenidas y la nota ponderada según el porcentaje asignado al rubro. Finalmente, suma todas las notas ponderadas para obtener la nota final del estudiante. El método también consulta la base de datos de MongoDB para obtener los nombres completos de los estudiantes, enriqueciendo así la información presentada. El resultado es una estructura detallada que muestra, para cada estudiante, sus calificaciones desglosadas por rubro y evaluación, así como su nota final.

En la **Figura 5** se puede apreciar el diagrama de flujo para este algoritmo.

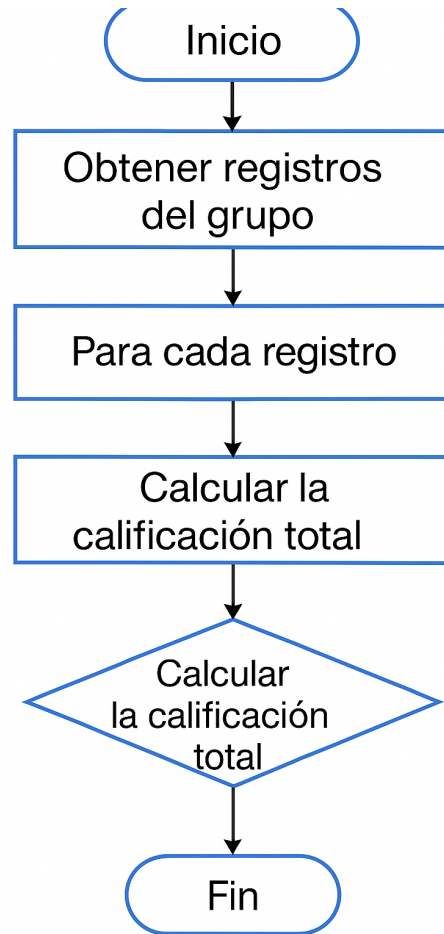


Figure 5: Diagrama de flujo para el cálculo de notas por grupo

1.1.6 Cálculo de Notas por Estudiante en un Grupo

Este algoritmo permite obtener el detalle de las calificaciones de un estudiante específico en un grupo y curso determinados. Realiza una consulta SQL que recupera todas las evaluaciones y calificaciones del estudiante, agrupadas por rubro. Para cada rubro, calcula el promedio de las calificaciones y la nota ponderada, considerando el porcentaje del rubro. El método estructura la información para mostrar, de manera clara, las evaluaciones, notas individuales, promedios y la nota final ponderada del estudiante. Además, consulta la base de datos de MongoDB para obtener el nombre completo del estudiante, proporcionando así un reporte personalizado y detallado.

1.1.7 Carga y Almacenamiento de Archivos en el Servidor

El sistema implementa un algoritmo para la gestión eficiente de archivos subidos por los usuarios, como documentos de entregas o recursos de curso. Cuando se recibe un archivo,

el método determina la ruta de almacenamiento adecuada en el servidor, organizando los archivos en carpetas jerárquicas basadas en el semestre, curso, grupo y carpeta específica. Antes de guardar el archivo, el algoritmo sanitiza el nombre de la carpeta para evitar caracteres no permitidos y crea la estructura de directorios si no existe. Finalmente, almacena el archivo físico en el servidor y retorna la ruta de acceso, asegurando la trazabilidad y organización de los documentos.

En la **Figura 6** se puede apreciar el diagrama de flujo para este algoritmo.

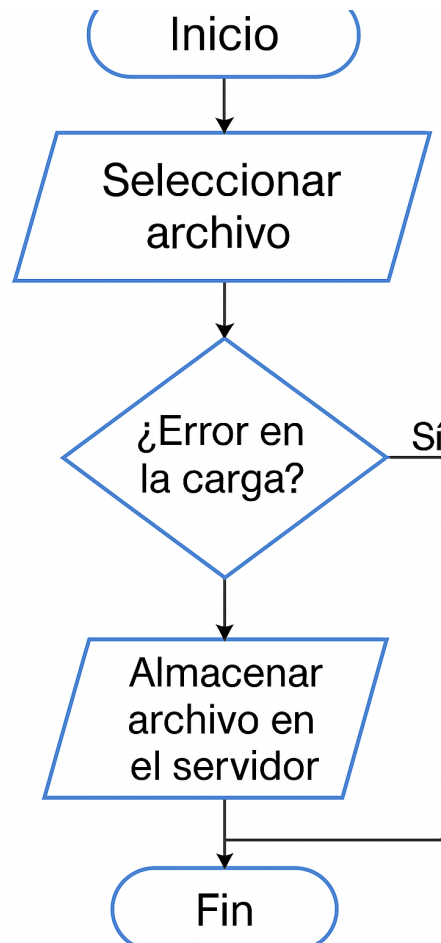


Figure 6: Diagrama de flujo para la carga y almacenamiento de archivos

1.1.8 Creación de Carpetas y Rubros por Defecto al Inicializar un Grupo

Cuando se crea un nuevo grupo académico, el sistema ejecuta un algoritmo que genera automáticamente un conjunto de carpetas y rubros estándar. Este proceso garantiza que cada grupo cuente con carpetas para presentaciones, quices, exámenes y proyectos, así como rubros de evaluación con porcentajes predefinidos. El método inserta estas entidades en la base de datos de manera transaccional, asegurando que el grupo esté listo para su uso

inmediato y que la estructura académica sea consistente en todos los cursos.

1.1.9 Asignación de Personas a Grupos

Este método gestiona la asignación de estudiantes y profesores a los grupos académicos. Para cada asignación, el algoritmo valida la existencia previa de la persona en la base de datos y verifica que no esté ya asignada al grupo correspondiente. Si la asignación es válida, la persona se asocia al grupo mediante las tablas de relación adecuadas. El proceso maneja tanto estudiantes como profesores y reporta cualquier error encontrado, como personas inexistentes o roles inválidos.

1.1.10 Obtención de Resumen de Calificaciones por Rubro

El sistema incluye un método que permite obtener un resumen de las calificaciones acumuladas por rubro para un estudiante en un curso determinado. El algoritmo recorre todos los rubros del curso, identifica las evaluaciones asociadas a cada rubro y suma las calificaciones obtenidas por el estudiante en dichas evaluaciones. El resultado es una lista que muestra, para cada rubro, la calificación acumulada, facilitando así el análisis del desempeño académico por criterios de evaluación.

1.2 Fronted

1.2.1 Gestión Dinámica de Rubros y Evaluaciones

En el frontend, se implementa un algoritmo interactivo para la gestión de rubros y evaluaciones desde la interfaz de usuario. El usuario puede crear, editar y eliminar rubros, así como asignar evaluaciones a cada rubro. El sistema valida en tiempo real que la suma de los porcentajes de los rubros no exceda el 100%, y que las evaluaciones dentro de cada rubro respeten el porcentaje máximo permitido. Además, el algoritmo permite la edición en línea de nombres y pesos, mostrando alertas y ajustes automáticos si se exceden los límites. Esta lógica asegura la coherencia de la estructura de evaluación antes de ser enviada al backend.

1.2.2 Carga Masiva de Datos desde Archivos Excel

El frontend proporciona un método para la carga masiva de estudiantes, profesores, cursos y grupos a partir de archivos Excel. El algoritmo permite al usuario seleccionar un archivo, valida su formato y lo envía al backend para su procesamiento. Durante la carga, el sistema muestra mensajes de progreso y notificaciones de éxito o error, facilitando la gestión eficiente de grandes volúmenes de datos académicos.

1.2.3 Visualización y Exportación de Reportes en PDF

El sistema implementa algoritmos para la generación y exportación de reportes académicos en formato PDF. Utilizando librerías de renderizado, el frontend captura la visualización de tablas de notas y listas de estudiantes, las convierte en imágenes y las inserta en documentos PDF. El usuario puede descargar estos reportes con un solo clic, obteniendo documentos formateados profesionalmente que incluyen cabeceras, tablas y totales, facilitando la documentación y el análisis fuera del sistema.

1.2.4 Gestión de Entregas y Retroalimentación

El frontend incluye un método avanzado para la gestión de entregas de evaluaciones. Los profesores pueden visualizar todas las entregas realizadas por los estudiantes, calificar cada una, añadir comentarios y subir archivos de retroalimentación. El algoritmo permite la edición directa de notas y comentarios, valida los valores ingresados y gestiona la subida de archivos de manera asíncrona. Además, permite alternar el estado de publicación de las calificaciones, asegurando que los estudiantes solo vean sus notas cuando el profesor lo decida.

En la **Figura 7** se puede apreciar el diagrama de flujo para este algoritmo.

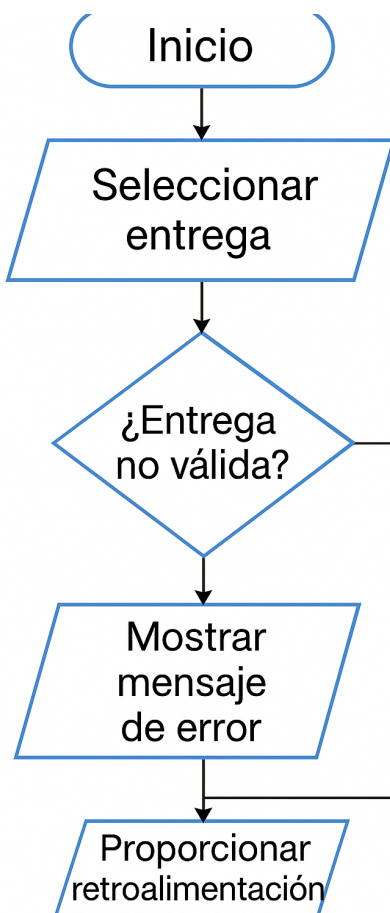


Figure 7: Diagrama de flujo para la gestión de entregas y retroalimentación

1.2.5 Gestión de Grupos y Minigrupos

El sistema ofrece una interfaz interactiva para la creación y administración de grupos y minigrupos. El algoritmo permite al usuario crear nuevas categorías de grupo, asignar estudiantes a minigrupos y editar la composición de los mismos. Se valida que no existan duplicados y que los estudiantes no estén asignados a más de un minigrupo dentro de la misma categoría. La lógica de la interfaz facilita la visualización jerárquica de grupos, categorías y miembros, permitiendo una gestión eficiente de actividades colaborativas.

1.2.6 Carga y Visualización de Documentos

El frontend implementa un método para la carga y visualización de documentos académicos. Los usuarios pueden subir archivos a carpetas específicas, ver la lista de documentos disponibles y descargar los archivos cuando lo requieran. El algoritmo gestiona la subida de archivos mediante formularios, muestra el progreso de la carga y actualiza la lista de documentos en

tiempo real. Además, permite la creación y eliminación de carpetas personalizadas, asegurando una organización flexible de los recursos.

1.2.7 Visualización de Noticias y Comunicados

El sistema incluye un algoritmo para la visualización y gestión de noticias y comunicados dentro de los grupos académicos. Los profesores pueden crear, editar y eliminar noticias, mientras que los estudiantes pueden visualizar los comunicados relevantes para sus cursos y grupos. El algoritmo organiza las noticias por fecha y relevancia, mostrando notificaciones y resúmenes en la interfaz principal.

1.2.8 Gestión de Evaluaciones y Asignaciones Grupales

El frontend proporciona una lógica para la creación y edición de evaluaciones, permitiendo asociarlas a rubros y, en caso de trabajos grupales, a categorías de grupo específicas. El algoritmo valida que las evaluaciones estén correctamente configuradas, gestiona la relación entre evaluaciones y minigrupos, y permite la subida de archivos de instrucciones. Además, facilita la edición de fechas, pesos y descripciones, asegurando que la información esté siempre actualizada y coherente con la estructura académica definida.

1.2.9 Visualización Detallada de Calificaciones para Estudiantes

El sistema implementa un método que permite a los estudiantes visualizar sus calificaciones de manera detallada, desglosadas por rubro y evaluación. El algoritmo calcula la nota final ponderada, muestra el estado de cada entrega, las notas obtenidas y la retroalimentación recibida. Además, permite acceder a los archivos de retroalimentación y visualizar comentarios personalizados, brindando una experiencia transparente y enriquecedora para el seguimiento académico individual.

2 Descripción de las Estructuras de Datos Desarrolladas

2.1 Backend

El backend utiliza modelos de datos en C# que representan las entidades principales del sistema académico. Entre las estructuras más relevantes se encuentran:

2.1.1 Grupo

La entidad Grupo es el núcleo de la organización académica. Representa un grupo de estudiantes y profesores que comparten un curso en un semestre específico. Cada grupo tiene un identificador único, un número de grupo (que puede repetirse en diferentes cursos o semestres), y un estado que indica si está activo o inactivo. Además, almacena referencias al semestre y al curso al que pertenece, permitiendo así la trazabilidad de la oferta académica a lo largo del tiempo. Un grupo mantiene relaciones con múltiples profesores (a través de la tabla de relación *ProfesorXGrupo*), estudiantes (*EstudianteXGrupo*), rubros de evaluación, carpetas de documentos, noticias y categorías de grupo para actividades colaborativas. Esta estructura permite gestionar de manera eficiente la composición y dinámica de cada grupo académico.

2.1.2 Carpeta

La estructura Carpeta modela un contenedor lógico de documentos dentro de un grupo. Cada carpeta tiene un identificador único y un nombre descriptivo, y está asociada a un grupo específico. Opcionalmente, puede estar vinculada a un profesor, lo que permite distinguir entre carpetas creadas por la administración del curso y aquellas generadas por los docentes. Cada carpeta puede contener múltiples documentos, facilitando la organización de materiales, entregas y recursos didácticos. Esta estructura es fundamental para mantener el orden y la accesibilidad de los archivos en el sistema.

2.1.3 Documento

La entidad Documento representa un archivo digital almacenado en una carpeta. Incluye atributos como el identificador, nombre del archivo, tamaño en bytes, fecha de subida y la carpeta a la que pertenece. Esta estructura permite gestionar de manera eficiente los recursos digitales, asegurando su correcta asociación con el contexto académico (grupo, carpeta, profesor). Además, facilita la trazabilidad y auditoría de los documentos subidos por estudiantes y profesores.

2.1.4 Estudiante

La estructura Estudiante almacena la información personal y académica de cada alumno. Incluye el carnet (identificador único), nombre, correo electrónico, teléfono y otros datos relevantes. Los estudiantes pueden estar asociados a varios grupos y minigrupos, lo que les permite participar en diferentes cursos y actividades colaborativas. Esta entidad es esencial para la gestión de la matrícula, el seguimiento académico y la comunicación institucional.

2.1.5 Profesor

La entidad Profesor contiene los datos de los docentes, como la cédula (identificador único), nombre, correo electrónico y teléfono. Los profesores pueden estar asignados a uno o más grupos, lo que les permite impartir varios cursos o colaborar en diferentes asignaturas. Esta estructura facilita la gestión de la carga académica, la asignación de responsabilidades y la comunicación entre docentes y estudiantes.

2.1.6 Rubro

El Rubro es una estructura clave para la evaluación académica. Representa un criterio o componente de evaluación dentro de un grupo, como exámenes, proyectos o quices. Cada rubro tiene un nombre, un porcentaje de ponderación (que contribuye a la nota final del curso) y está asociado a un grupo específico. Los rubros pueden contener múltiples evaluaciones, permitiendo una evaluación detallada y justa del desempeño estudiantil. Esta estructura asegura la transparencia y flexibilidad en la definición de los esquemas de evaluación.

2.1.7 Evaluacion

La estructura Evaluacion modela una actividad evaluativa concreta, como un examen parcial, un proyecto o una presentación. Incluye atributos como el nombre, peso dentro del rubro, fecha y hora de entrega, tipo de evaluación (individual o grupal), archivo de especificación (instrucciones) y referencias al rubro y la categoría a la que pertenece. Esta entidad permite gestionar el calendario de evaluaciones, las instrucciones para los estudiantes y la relación con los criterios de evaluación definidos por los docentes.

2.1.8 Entregable

La entidad Entregable representa una entrega realizada por un estudiante para una evaluación específica. Contiene la fecha y hora de entrega, el archivo entregado, la evaluación correspondiente y el estudiante que realizó la entrega. Esta estructura es fundamental para el seguimiento de las actividades académicas, la gestión de plazos y la evaluación del desempeño individual o grupal.

2.1.9 Nota

La estructura Nota almacena la calificación obtenida por un estudiante en una entrega, junto con observaciones del docente y el estado de la nota (publicada o no). Permite asociar

cada nota a un entregable y a un estudiante, facilitando la generación de reportes, el análisis del rendimiento y la retroalimentación personalizada.

2.1.10 Noticia

La entidad Noticia permite la publicación de mensajes, anuncios o comunicados dentro de un grupo académico. Incluye el título, fecha de publicación, mensaje, grupo destinatario y el profesor responsable de la publicación. Esta estructura facilita la comunicación institucional y la difusión de información relevante para el desarrollo del curso.

2.1.11 Semestre

La estructura Semestre representa un periodo académico, definido por el año, el periodo (primer semestre, segundo semestre o verano) y el estado (activo o inactivo). Permite organizar la oferta académica, gestionar la matrícula y planificar las actividades a lo largo del tiempo.

2.1.12 CategoriaGrupo

La entidad CategoriaGrupo permite clasificar los minigrupos dentro de un grupo principal. Cada categoría tiene un nombre y está asociada a un grupo, facilitando la organización de actividades colaborativas, proyectos o trabajos en equipo. Esta estructura aporta flexibilidad y orden a la gestión de grupos grandes.

2.1.13 MiniGrupo

La estructura MiniGrupo representa un subgrupo de estudiantes dentro de una categoría de grupo. Incluye un identificador, nombre y referencia a la categoría correspondiente. Los minigrupos son utilizados para trabajos colaborativos, permitiendo una gestión eficiente de equipos dentro de cursos con muchos estudiantes.

2.2 Fronted

En el frontend, desarrollado en React y TypeScript, se definen interfaces y tipos que reflejan las entidades del backend y facilitan la gestión de datos en la aplicación. Las estructuras más importantes son:

2.2.1 MainGroup

La estructura MainGroup representa un grupo principal en la interfaz de usuario. Incluye el identificador único, número de grupo, año, periodo, código de curso, estado y la lista de profesores asociados. Permite mostrar y gestionar la información esencial de cada grupo académico, facilitando la navegación y administración desde el frontend.

2.2.2 Professor

La entidad Professor almacena los datos de un profesor, tales como cédula, nombre, correo y teléfono. Esta estructura es utilizada para mostrar la información de los docentes en las vistas de grupos, cursos y reportes, así como para gestionar la asignación de profesores a grupos y cursos.

2.2.3 Student

La estructura Student contiene la información de un estudiante, incluyendo carnet, cédula, nombre, correo y teléfono. Es utilizada en la visualización de listas de estudiantes, reportes, asignación a grupos y minigrupos, y en la gestión de entregas y calificaciones.

2.2.4 Course

La entidad Course representa un curso académico, con atributos como código, nombre, cantidad de créditos, horas lectivas y estado. Permite mostrar la oferta académica, gestionar la matrícula y organizar los grupos y evaluaciones asociados a cada curso.

2.2.5 Folder

La estructura Folder modela una carpeta de documentos en la interfaz, asociada a un grupo y, opcionalmente, a un profesor. Permite organizar los archivos subidos por estudiantes y docentes, facilitando la navegación y búsqueda de recursos.

2.2.6 File

La entidad File representa un archivo almacenado en una carpeta, incluyendo identificador, nombre, tamaño, fecha de subida y la carpeta a la que pertenece. Es utilizada para mostrar la lista de documentos disponibles, gestionar descargas y eliminar archivos cuando sea necesario.

2.2.7 Entrega

La estructura Entrega representa una entrega de evaluación realizada por un estudiante o grupo. Incluye información sobre la evaluación, si es grupal, estado de entrega, calificación, comentarios y retroalimentación. Permite mostrar el historial de entregas, gestionar la subida de archivos y visualizar la retroalimentación recibida.

2.2.8 Rubro

La entidad Rubro representa un criterio de evaluación, con identificador, nombre, porcentaje y la lista de evaluaciones asociadas. Es utilizada para mostrar la estructura de evaluación del curso, calcular notas y validar la coherencia de los porcentajes asignados.

2.2.9 Evaluacion

La estructura Evaluacion modela una evaluación específica, incluyendo nombre, descripción, porcentaje, fecha y hora de entrega, si es grupal y el documento de instrucciones. Permite gestionar el calendario de evaluaciones, mostrar detalles y facilitar la entrega de trabajos.

2.2.10 Category (Grupo)

La entidad Category permite clasificar minigrupos dentro de un grupo principal, facilitando la organización de actividades colaborativas y la asignación de estudiantes a equipos de trabajo.

2.2.11 Minigroup

La estructura Minigroup representa un subgrupo de estudiantes, con identificador, categoría, nombre y la lista de estudiantes que lo conforman. Es utilizada para gestionar equipos de trabajo, asignar entregas grupales y visualizar la composición de los grupos.

2.2.12 News (Noticia)

La entidad News permite mostrar noticias o anuncios en la interfaz, incluyendo título, fecha, autor y cuerpo del mensaje. Facilita la comunicación entre profesores y estudiantes, mostrando información relevante y actualizada.

2.2.13 Semestre

La estructura Semestre representa un periodo académico, con identificador, año, periodo y estado. Permite organizar la información por ciclos lectivos y filtrar los datos según el semestre seleccionado.

2.2.14 GradeCategory

La entidad GradeCategory agrupa las calificaciones por rubro, mostrando el porcentaje, promedio, nota ponderada y las evaluaciones asociadas. Facilita la visualización detallada del desempeño académico y el cálculo de la nota final.

2.2.15 StudentGrades

La estructura StudentGrades almacena las calificaciones de un estudiante, incluyendo carnet, nombre, lista de categorías de notas y la nota total. Es utilizada en reportes, visualización de notas y análisis del rendimiento académico.

2.2.16 Nota

La entidad Nota representa una nota individual obtenida en una evaluación específica. Permite mostrar el detalle de las calificaciones, calcular promedios y generar reportes personalizados.

2.2.17 Otros

Existen otras estructuras auxiliares en el frontend para la gestión de reportes, cargas de archivos, relaciones entre entidades y visualización de datos. Estas estructuras complementan la funcionalidad principal del sistema, optimizando la experiencia del usuario y la integración con el backend.

3 Problemas Conocidos

Durante la realización del proyecto CEDigital, no se encontró algún problema sin solución hasta el momento.

4 Problemas Encontrados

4.1 Backend

4.1.1 Conexión Intermitente entre Entity Framework y SQL Server

- **Descripción:** Durante el desarrollo del backend, se presentó un problema intermitente en la conexión entre Entity Framework (EF Core) y la base de datos SQL Server. Al ejecutar consultas o al intentar migrar cambios, el sistema arrojaba errores como `A network-related or instance-specific error occurred while establishing a connection to SQL Server` o simplemente se colgaba sin dar respuesta. Este fallo impedía compilar y probar funcionalidades del sistema, especialmente durante la implementación de controladores CRUD como los de cursos, carreras y semestres.
- **Intentos de Solución sin Éxito:** Se revisaron las cadenas de conexión múltiples veces y se intentó modificar el puerto y la IP del servidor en `appsettings.json`. También se intentó cambiar la versión del paquete de EF Core y reiniciar manualmente los servicios de SQL Server en local, pero los errores persistían de forma esporádica.
- **Soluciones Encontradas:** El problema se resolvió configurando correctamente el firewall de Windows para permitir conexiones entrantes al puerto utilizado por SQL Server (1433 por defecto), y asegurando que el servidor SQL estuviera configurado en modo mixto (autenticación SQL + Windows). Adicionalmente, se configuró un tiempo de espera mayor en la cadena de conexión utilizando el parámetro `Connect Timeout=30`.
- **Recomendaciones:** Es recomendable validar que el servidor SQL esté corriendo antes de iniciar el backend y utilizar herramientas como `sqlcmd` o SQL Server Management Studio para verificar que la conexión sea estable. Además, se debe asegurar que el puerto usado esté habilitado en el firewall.
- **Conclusiones:** Este problema muestra la importancia de considerar aspectos de infraestructura incluso en proyectos locales. Muchas fallas que parecen estar en el código realmente se deben a problemas de configuración del entorno.
- **Bibliografía Consultada:** <https://stackoverflow.com/questions/60045984/entity-framework>
<https://learn.microsoft.com/en-us/sql/reporting-services/report-server/configure-a-view=sql-server-ver17>

4.1.2 API con Métodos Confusos por Colisión de Rutas

- **Descripción:** Durante la implementación del controlador `GrupoController`, se definieron múltiples métodos GET con rutas similares como `GET /api/grupo/id.semestre/codigo_curso` y `GET /api/grupo/id`, lo que ocasionó ambigüedades en el ruteo de ASP.NET Core. Al hacer pruebas desde Postman o el navegador, las peticiones a `/api/grupo/1` eran redirigidas incorrectamente o generaban errores HTTP 404 o 500, dependiendo del orden en que se registraban los métodos.
- **Intentos de Solución sin Éxito:** Inicialmente se intentó cambiar el orden de declaración de los métodos en el código fuente, creyendo que ASP.NET elegiría la primera coincidencia encontrada. También se trató de hacer el casteo explícito de los parámetros en los atributos de ruta para evitar confusión, pero esto no resolvió el conflicto entre rutas similares con diferente número de parámetros.
- **Soluciones Encontradas:** La solución fue modificar la ruta del segundo método para que tuviera un prefijo distintivo, por ejemplo, cambiando `[HttpGet("{id.semestre}/{codigo_curso}")]` a `[HttpGet("by-semester-curso/{id.semestre}/{codigo_curso}")]` con lo cual se evitó la colisión con la ruta genérica por ID. Esta técnica es recomendada por Microsoft para prevenir ambigüedades en rutas sobrecargadas.
- **Recomendaciones:** Se recomienda usar prefijos semánticos cuando se tengan rutas similares con parámetros múltiples. Además, es buena práctica definir rutas explícitas y únicas para cada acción del controlador y documentarlas en Swagger o Postman para evitar confusión.
- **Conclusiones:** Las colisiones de rutas pueden provocar errores difíciles de rastrear y deben ser tratadas desde el diseño inicial del API. Un nombre claro y explícito en las rutas mejora la mantenibilidad y el consumo del servicio.
- **Bibliografía Consultada:** <https://stackoverflow.com/questions/66551109/asp-net-core-api-routes-conflict>
<https://learn.microsoft.com/en-us/aspnet/core/web-api/action-return-types>

4.1.3 Desincronización entre Entidades Relacionadas en Entity Framework

- **Descripción:** Al trabajar con entidades relacionadas como `Curso`, `Grupo` y `Semestre`, se presentó un problema de desincronización cuando se hacían operaciones encadenadas. Por ejemplo, al crear un grupo vinculado a un semestre y a un curso, los datos

de las relaciones no se reflejaban correctamente en la base de datos. Esto se manifestaba como campos nulos o duplicados en la tabla intermedia, y provocaba errores al consultar o serializar los datos desde la API.

- **Intentos de Solución sin Éxito:** Se intentó mapear manualmente las relaciones dentro del modelo usando anotaciones de data (`['ForeignKey']`, `['InverseProperty']`), y también se probó con `Include()` en las consultas para forzar la carga de entidades relacionadas. Sin embargo, los problemas persistían debido a inconsistencias en la configuración de las claves foráneas y el seguimiento de cambios de EF Core.
- **Soluciones Encontradas:** La solución fue revisar completamente los modelos de datos y establecer explícitamente las relaciones en el archivo `AppDbContext.cs` utilizando Fluent API, lo que permitió definir con precisión la cardinalidad y las claves foráneas entre entidades. Además, se ajustó el uso del método `AddAsync()` para que asignara correctamente los IDs de entidades relacionadas antes de ejecutar `SaveChangesAsync()`.
- **Recomendaciones:** Es recomendable configurar las relaciones entre entidades utilizando Fluent API en lugar de depender exclusivamente de las anotaciones en los modelos. También se debe tener cuidado con el orden de las operaciones al guardar entidades relacionadas, asegurando que las dependencias ya existan o estén correctamente instanciadas.
- **Conclusiones:** Entity Framework requiere atención a los detalles en relaciones entre entidades. Malas configuraciones pueden derivar en pérdida de integridad referencial y errores de lógica difíciles de detectar.
- **Bibliografía Consultada:** <https://learn.microsoft.com/en-us/ef/core/modeling/relationships>

4.1.4 Códigos de Estado Incorrectos en Respuestas HTTP

- **Descripción:** Durante la fase de prueba de la API, se detectó que algunos endpoints devolvían códigos de estado HTTP inapropiados. Por ejemplo, el método `POST` en el `CursoController` retornaba un `200 OK` en lugar de `201 Created`, y los métodos `PUT` y `DELETE` respondían con `200 OK` cuando debían retornar `204 No Content`. Este comportamiento no afectaba el funcionamiento básico, pero incumplía con las convenciones REST y dificultaba el desarrollo frontend al no reflejar claramente el resultado de las operaciones.

- **Intentos de Solución sin Éxito:** Inicialmente se intentó forzar manualmente los códigos usando `return StatusCode(201, objeto)` o `return StatusCode(204)`, sin utilizar los métodos estándar como `CreatedAtAction` o `NoContent`. Sin embargo, esto generaba respuestas poco consistentes y omitía encabezados importantes como la ubicación del recurso creado.
- **Soluciones Encontradas:** La solución fue utilizar los métodos específicos provistos por ASP.NET Core: `CreatedAtAction` para POST, `NoContent()` para DELETE y PUT, y `BadRequest()`, `NotFound()`, `Unauthorized()` según correspondiera en cada caso. Además, se revisó toda la API y se ajustaron las respuestas para que coincidieran con los estados esperados según la operación realizada.
- **Recomendaciones:** Es fundamental seguir las convenciones REST al diseñar una API, utilizando los métodos de respuesta estándar que ofrece el framework. Esto no solo mejora la legibilidad del código, sino que también facilita la integración con el frontend y el uso de herramientas como Swagger o Postman.
- **Conclusiones:** Una API que responde con los códigos HTTP correctos transmite mayor claridad y profesionalismo. Este tipo de errores, aunque sutiles, pueden afectar pruebas automatizadas, herramientas cliente y la experiencia del usuario final.
- **Bibliografía Consultada:** <https://learn.microsoft.com/en-us/aspnet/core/web-api/action-return-types>, <https://stackoverflow.com/questions/62323734/how-to-return-crea>

4.2 Fronted

4.2.1 Conflictos entre Bootstrap y Estilos Personalizados

- **Descripción:** Al integrar Bootstrap con el proyecto desarrollado en Next.js y TypeScript, surgieron conflictos visuales y de comportamiento entre los estilos proporcionados por Bootstrap y los estilos personalizados definidos en archivos CSS locales. Componentes como formularios, botones y tarjetas no se mostraban como se esperaba, y en varios casos el diseño se rompía al aplicar clases personalizadas junto con clases de Bootstrap.
- **Intentos de Solución sin Éxito:** Inicialmente se intentó aplicar las clases personalizadas directamente sobre las clases de Bootstrap en el JSX, esperando que estas sobrescribieran el estilo por cascada. También se intentó reorganizar el orden de importación de los archivos CSS y de Bootstrap en el `_app.tsx`, pero el problema persistía, especialmente con componentes anidados.

- **Soluciones Encontradas:** Se identificó que Bootstrap utiliza reglas CSS muy específicas que sobrescriben fácilmente los estilos personalizados si no se emplean selectores más fuertes o no se aplican correctamente los principios de especificidad. La solución fue encapsular los estilos personalizados por componente utilizando archivos CSS módulos (`.module.css`) y aplicar nombres de clase únicos para evitar colisiones. Además, se empleó la estrategia de importar Bootstrap globalmente primero y luego los estilos locales en cada componente, asegurando un orden de carga correcto.
- **Recomendaciones:** Al trabajar con Bootstrap, se recomienda usar CSS Modules para encapsular estilos y evitar clases genéricas que puedan colisionar. También es importante mantener el orden correcto de importación y limitar el uso de clases Bootstrap cuando se requiere un diseño personalizado complejo.
- **Conclusiones:** La combinación de librerías de estilos como Bootstrap con estilos propios puede causar efectos secundarios inesperados si no se organiza correctamente el manejo del CSS. Utilizar buenas prácticas como CSS Modules y respetar la especificidad del CSS permite mantener un diseño coherente y mantenible.
- **Bibliografía Consultada:** <https://stackoverflow.com/questions/72410147/how-to-use-nextjs-css-modules>
<https://nextjs.org/docs/13/app/building-your-application/styling/css-modules>

4.2.2 Problemas al Consumir APIs con Tipos Incorrectos en TypeScript

- **Descripción:** Durante el desarrollo de los servicios que consumen los endpoints del backend en la aplicación frontend con TypeScript, surgieron errores de tipo al intentar mapear las respuestas de las APIs a interfaces definidas localmente. Por ejemplo, se definían tipos como `Curso` o `Grupo` en los archivos de `types/`, pero al momento de recibir la respuesta, TypeScript arrojaba errores como `Type 'any' is not assignable to type 'Curso[]'` o bien advertencias de posibles campos `undefined`. Esto impedía compilar el proyecto correctamente y dificultaba la integración entre backend y frontend.
- **Intentos de Solución sin Éxito:** Inicialmente se intentó desactivar temporalmente la verificación de tipos estricta en el archivo `tsconfig.json`, lo cual resolvía los errores pero comprometía la seguridad del código. También se intentó forzar el tipado con `as Tipo`, pero esto generaba fallos en tiempo de ejecución al recibir datos inesperados desde la API.

- **Soluciones Encontradas:** La solución consistió en definir cuidadosamente las interfaces en los archivos de `types/` y usar el método `axios.get<Tipo>` para tipar correctamente las respuestas desde el cliente. Además, se validaron los datos recibidos antes de procesarlos en la interfaz, asegurando que el contenido fuera congruente con los tipos definidos. También se agregó un archivo centralizado para las funciones de fetch (`api.ts`) para unificar la gestión de respuestas y errores.
- **Recomendaciones:** Se recomienda evitar usar `any` y preferir la definición estricta de interfaces para todos los datos esperados desde la API. Usar herramientas como Axios con tipado genérico ayuda a mantener la integridad del código y evitar errores sutiles. Validar las respuestas en tiempo de ejecución también es una buena práctica.
- **Conclusiones:** TypeScript aporta gran seguridad, pero también exige precisión en la definición y uso de tipos. Es mejor invertir tiempo en diseñar bien las interfaces que permitir errores en tiempo de ejecución por tipado deficiente.
- **Bibliografía Consultada:** <https://blog.logrocket.com/how-to-make-http-requests-like-a>

4.2.3 Fallos en la Navegación al Usar Link en lugar de Router.push en Next.js

- **Descripción:** Al implementar la navegación entre vistas dentro de la aplicación desarrollada en Next.js, surgieron errores en tiempo de ejecución al utilizar el componente `<Link>` en botones o elementos no compatibles. Esto generaba advertencias como `Warning: validateDOMNesting(...): <a> cannot appear as a descendant of <button>` o simplemente no realizaba la navegación esperada. Este problema se presentó al redirigir a vistas específicas por rol, como las vistas de profesor o estudiante, inmediatamente después del login.
- **Intentos de Solución sin Éxito:** Inicialmente se intentó envolver botones con el componente `<Link>` o utilizarlo dentro de `onClick`, lo cual no está soportado por Next.js de forma limpia. También se intentó manipular el DOM directamente con `window.location.href`, pero esto provocaba recargas completas de página y pérdida del estado de sesión.
- **Soluciones Encontradas:** La solución fue utilizar el hook `useRouter()` provisto por Next.js y redirigir mediante `router.push('/ruta')` dentro de funciones `onClick`. Esto permitió navegar de forma programática sin errores y sin recargar la página. Además, se refactorizaron los componentes de navegación para separar la lógica de renderizado visual del comportamiento de navegación.

- **Recomendaciones:** Es recomendable usar `<Link>` solo con elementos `<a>` y preferir `router.push()` para la navegación programática desde botones, funciones o eventos de login. También se debe evitar mezclar comportamientos visuales con lógica de navegación dentro del mismo componente.
- **Conclusiones:** Conocer los límites del enrutamiento en Next.js evita errores comunes de navegación. Utilizar correctamente los hooks como `useRouter()` permite una experiencia más fluida y limpia.
- **Bibliografía Consultada:** <https://nextjs.org/docs/api-reference/next/router#userouter>

5 Planeamiento de Actividades

5.1 Plan de Trabajo

En la presente sección, se describen en forma tabular, el plan de trabajo realizado en el proyecto para cada equipo de trabajo.

5.1.1 Backend

En la **Tabla 1** se puede apreciar el plan de trabajo para el equipo de backend.

Actividad	Fecha de entrega	Responsable
Crear repositorio backend	24 de abril	Adrián Muñoz
Configurar Azure DevOps	25 de abril	Jimmy Feng, Adrián Muñoz
Estructura base en C#	26 de abril	Jimmy Feng
Conexión SQL y MongoDB	27 de abril	Alexander Montero
Configurar Swagger API	28 de abril	Jimmy Feng
Crear modelo Curso	29 de abril	Adrián Muñoz
CRUD cursos completo	30 de abril	Jimmy Feng
CRUD carreras básico	1 de mayo	Alexander Montero
CRUD semestres	2 de mayo	Adrián Muñoz
Validar periodo semestre	2 de mayo	Adrián Muñoz
CRUD grupos	3 de mayo	Jimmy Feng
Ruta grupos por curso	4 de mayo	Alexander Montero
Login y autenticación	5 de mayo	Jimmy Feng
Middleware de sesión	6 de mayo	Alexander Montero
Protección de rutas API	7 de mayo	Jimmy Feng, Alexander Montero
Modelo estudiante MongoDB	8 de mayo	Adrián Muñoz
CRUD estudiantes MongoDB	9 de mayo	Jimmy Feng
Modelo profesor MongoDB	10 de mayo	Alexander Montero
CRUD profesores MongoDB	11 de mayo	Jimmy Feng
Validar roles por sesión	12 de mayo	Adrián Muñoz
CRUD evaluaciones	13 de mayo	Jimmy Feng, Alexander Montero
Publicación de notas	14 de mayo	Adrián Muñoz
Generar PDF evaluaciones	15 de mayo	Alexander Montero
CRUD rubros evaluación	16 de mayo	Jimmy Feng
Relaciones curso-semestre	17 de mayo	Adrián Muñoz
Gestión carga académica	18 de mayo	Jimmy Feng
Depuración endpoints	19 de mayo	Alexander Montero
Pruebas postman y ajustes	20 de mayo	Adrián Muñoz, Jimmy Feng
Documentación Swagger API	21 de mayo	Alexander Montero
Cierre técnico backend	22 de mayo	Jimmy Feng

Table 1: Plan de trabajo del equipo de Backend (Adrián Muñoz, Jimmy Feng y Alexander Montero).

5.1.2 Fronted

En la **Tabla 2** se puede apreciar el plan de trabajo para el equipo de frontend.

Actividad	Fecha de entrega	Responsable
Crear repositorio frontend	24 de abril	José Barquero
Estructura inicial proyecto	25 de abril	Diego Salas
Organizar carpetas base	25 de abril	José Barquero, Diego Salas
Configurar React Router	26 de abril	Alexander Montero
Conectar API backend	27 de abril	Diego Salas
Estilos globales base	27 de abril	José Barquero
Pantalla login admin	28 de abril	Diego Salas
Pantalla login estudiante	29 de abril	José Barquero
Pantalla login profesor	29 de abril	Alexander Montero
Redirección por rol	30 de abril	Diego Salas
Vista admin cursos	1 de mayo	José Barquero
Vista crear semestre	2 de mayo	Alexander Montero
Carga Excel estudiantes	3 de mayo	José Barquero
Vista grupos admin	4 de mayo	Diego Salas
Vista profesor documentos	6 de mayo	Alexander Montero
Vista profesor noticias	7 de mayo	Diego Salas
Vista profesor evaluaciones	8 de mayo	José Barquero
Formulario rubros curso	9 de mayo	José Barquero
Formulario subir entregas	10 de mayo	Diego Salas
Vista estudiante carpetas	11 de mayo	Alexander Montero
Vista estudiante entregas	12 de mayo	Diego Salas
Vista estudiante noticias	13 de mayo	José Barquero
Vista estudiante notas	14 de mayo	Diego Salas
Integración subida archivos	15 de mayo	Alexander Montero
Gestión validación notas	16 de mayo	José Barquero
Descarga PDF notas	17 de mayo	Alexander Montero
Estilos componentes clave	18 de mayo	José Barquero
Pruebas navegación vistas	19 de mayo	Diego Salas
Refactorización de rutas	20 de mayo	Alexander Montero
Corrección bugs visuales	21 de mayo	Diego Salas
Cierre técnico frontend	22 de mayo	José Barquero

Table 2: Plan de trabajo del equipo de Frontend (José Barquero, Diego Salas y Alexander Montero).

5.2 Minutas de Sesiones de Trabajo

A continuación, se describen todas las minutas de las sesiones de trabajo realizada por todo el grupo.

5.2.1 Minuta #1 - 24 de abril

- **Hora de Inicio:** 7:00 pm
- **Hora de Fin:** 8:00 pm
- **Tema:** Discusión del modelo conceptual y relacional de datos
- **Participantes:** José Barquero, Diego Salas, Alexander Montero, Jimmy Feng, Adrián Muñoz
- **Cuerpo de Reunión:** En esta primera sesión del equipo se definió el punto de partida del proyecto CEDigital, enfocado principalmente en la definición del modelo conceptual y relacional. Se trabajó colaborativamente en el reconocimiento de las entidades clave del sistema: Curso, Semestre, Estudiante, Profesor, Evaluación, Grupo y Rubro de Evaluación. Se discutieron los atributos principales, identificadores primarios, claves foráneas y las relaciones entre entidades (uno a muchos, muchos a muchos). Además, se establecieron criterios para dividir qué datos irían en SQL Server (estructura académica) y cuáles en MongoDB (datos personales y autenticación). Se acordó que Jimmy Feng y Adrián Muñoz serían responsables de formalizar el modelo relacional en SQL y que José Barquero se encargaría de preparar el diagrama para el documento técnico. Se estableció como prioridad que cada modelo fuera consistente con los requerimientos del PDF del proyecto, considerando las futuras relaciones con los controladores API y los formularios del frontend. Se utilizó Google Meet como medio de comunicación y se acordó la siguiente reunión para revisar el modelo físico y comenzar con la estructura del backend.

5.2.2 Minuta #2 - 26 de abril

- **Hora de Inicio:** 8:00 pm
- **Hora de Fin:** 9:00 pm
- **Tema:** Asignación de tareas técnicas iniciales
- **Participantes:** José Barquero, Diego Salas, Alexander Montero, Jimmy Feng y Adrián Muñoz

- **Cuerpo de Reunión:** Esta sesión virtual se enfocó en definir y distribuir las primeras tareas técnicas del Sprint 1. Se acordó que José Barquero y Diego Salas se encargarían del repositorio frontend y de estructurar las carpetas base del proyecto Next.js, incluyendo configuración de ESLint, TypeScript, Bootstrap y estructura modular de componentes. Por otro lado, Jimmy Feng asumiría la creación del repositorio backend con Alexander Montero, configurando la solución inicial en .NET y conectando con SQL Server y MongoDB. También se discutió la integración temprana de Swagger para documentar las APIs desde el inicio del proyecto. Además, se validó la estructura de carpetas y convenciones de nombres para asegurar la consistencia entre ambos equipos. Se aclararon dudas sobre cómo sincronizar los controladores del backend con los servicios del frontend, y se estableció como prioridad dejar toda la base montada antes del cierre de la semana. La reunión se realizó por Google Meet y concluyó con acuerdos de comunicación interna mediante grupos de WhatsApp y actualizaciones por Azure DevOps.

5.2.3 Minuta #3 - 4 de mayo

- **Hora de Inicio:** 7:30 pm
- **Hora de Fin:** 8:30 pm
- **Tema:** Revisión de autenticación y validación de roles
- **Participantes:** Jimmy Feng, Alexander Montero, Adrián Muñoz
- **Cuerpo de Reunión:** En esta sesión se abordaron los avances y dificultades en la implementación del sistema de autenticación utilizando MongoDB, así como la validación de los roles de usuario (estudiante, profesor y administrador). Jimmy Feng presentó el estado del controlador de autenticación y el uso de **AuthService** para validar credenciales y retornar el rol correspondiente. Alexander Montero compartió el esquema actual de MongoDB para usuarios y las validaciones básicas aplicadas, mientras que Adrián Muñoz planteó dudas sobre cómo proteger rutas específicas en el backend dependiendo del rol autenticado. Se discutió el uso de middlewares en ASP.NET Core para interceptar los tokens o credenciales antes de permitir el acceso a rutas restringidas. También se acordó agregar validaciones adicionales para evitar accesos cruzados entre roles. Se enfatizó la importancia de documentar bien este comportamiento para que el frontend pueda redirigir correctamente según el rol retornado. Finalmente, se propuso como tarea inmediata centralizar la respuesta del login con campos consis-

tentes para facilitar el manejo desde React. La reunión se realizó por Google Meet y se definieron responsables para la integración completa en los próximos días.

5.2.4 Minuta #4 - 6 de mayo

- **Hora de Inicio:** 8:00 pm
- **Hora de Fin:** 9:00 pm
- **Tema:** Validación de formularios y carga de Excel
- **Participantes:** José Barquero, Alexander Montero, Adrián Muñoz
- **Cuerpo de Reunión:** En esta reunión se discutió la implementación de los formularios utilizados por el administrador para cargar datos académicos, específicamente la asignación de grupos y estudiantes a cursos mediante archivos Excel. Adrián Muñoz comentó las validaciones que ya se están aplicando del lado del backend para asegurar la consistencia de los datos antes de insertar en la base de datos. José Barquero presentó los avances en el diseño del formulario en React y los retos para validar los datos antes de enviarlos al backend, especialmente en cuanto al manejo de errores del archivo. Alexander Montero propuso una estructura de ejemplo de Excel con columnas mínimas obligatorias y se acordó incluir una funcionalidad para previsualizar el contenido antes de confirmar la carga. También se discutió el uso de bibliotecas como `xlsx` en el frontend para parsear el archivo localmente. Se dividieron tareas para asegurar que el flujo de carga por archivo funcionara de manera fluida antes del cierre del sprint. La reunión fue virtual por Google Meet.

5.2.5 Minuta #5 - 8 de mayo

- **Hora de Inicio:** 7:30 pm
- **Hora de Fin:** 8:30 pm
- **Tema:** Gestión de documentos y retroalimentación
- **Participantes:** Diego Salas, Alexander Montero, Jimmy Feng
- **Cuerpo de Reunión:** El objetivo de esta reunión fue revisar la implementación de la funcionalidad para que los profesores gestionen documentos dentro de sus cursos y retroalimenten a los estudiantes. Jimmy Feng explicó el endpoint desarrollado en el

backend para subir, editar y eliminar archivos asociados a un grupo, así como el almacenamiento de metadatos y la vinculación con MongoDB para registrar la identidad del usuario que realiza la acción. Diego Salas presentó el diseño de interfaz desde React, incluyendo la navegación por carpetas, la visualización de archivos y el formulario para subir nuevos recursos. Alexander Montero planteó la necesidad de permitir comentarios de retroalimentación en evaluaciones, por lo cual se acordó agregar un campo de observación en los registros de calificación. Se estableció como acuerdo definir una estructura de carpetas preestablecida por curso, para mantener el orden. También se identificó la necesidad de limitar el tamaño de archivos desde el backend. La reunión concluyó con tareas asignadas a cada integrante y se realizó por medio de Google Meet.

5.2.6 Minuta #6 - 11 de mayo

- **Hora de Inicio:** 8:00 pm
- **Hora de Fin:** 9:00 pm
- **Tema:** Publicación de notas y generación de reportes
- **Participantes:** José Barquero, Adrián Muñoz, Alexander Montero
- **Cuerpo de Reunión:** Esta sesión se centró en definir el proceso para que los profesores puedan publicar las calificaciones de los estudiantes y generar reportes en formato PDF. Adrián Muñoz explicó el diseño del endpoint en el backend que registra las notas por evaluación, así como la estructura del objeto JSON que se almacena en base a la rubrica definida previamente. José Barquero mostró cómo se representará esa información en el frontend del estudiante, permitiendo que cada uno vea su nota de forma individual, junto con observaciones, en una tabla clara y responsiva. Alexander Montero propuso el uso de una biblioteca para generación de PDFs desde el backend, lo cual fue aceptado como solución para generar documentos que puedan ser descargados por los profesores. También se discutieron aspectos como el ordenamiento por rubros, el cálculo automático del promedio y la posibilidad de incluir un encabezado institucional. Se definieron responsables para implementar cada parte y se aclararon las necesidades de sincronización entre ambos equipos. La reunión fue virtual mediante Google Meet.

5.2.7 Minuta #7 - 15 de mayo

- **Hora de Inicio:** 7:00 pm

- **Hora de Fin:** 8:00 pm
- **Tema:** Validación de entregas y noticias del estudiante
- **Participantes:** Diego Salas, José Barquero, Alexander Montero
- **Cuerpo de Reunión:** En esta sesión se abordó el comportamiento de la vista del estudiante en cuanto al manejo de entregas de evaluaciones y la lectura de noticias asociadas a sus cursos. Diego Salas explicó la lógica implementada en la vista de entregas, donde se permite al estudiante subir archivos por grupo y se valida que no existan duplicados. José Barquero propuso mejoras en la experiencia de usuario, como la indicación visual del estado de entrega (pendiente, entregado, calificado). Alexander Montero destacó la importancia de mostrar mensajes informativos cuando un curso no tenga evaluaciones activas o noticias publicadas. Se definieron estructuras comunes para los objetos de entrega y noticia para facilitar el reuso de componentes en el frontend. Además, se evaluó la posibilidad de implementar una ordenación cronológica inversa de las noticias, para mostrar primero las más recientes. Se repartieron tareas para finalizar la funcionalidad antes del cierre del sprint y se propuso realizar pruebas de integración con datos reales simulados. La sesión fue realizada mediante Google Meet.

5.2.8 Minuta #8 - 18 de mayo

- **Hora de Inicio:** 8:00 pm
- **Hora de Fin:** 9:00 pm
- **Tema:** Integración final y pruebas generales
- **Participantes:** José Barquero, Diego Salas, Jimmy Feng, Alexander Montero y Adrián Muñoz.
- **Cuerpo de Reunión:** Esta reunión tuvo como objetivo coordinar el proceso de integración final entre los módulos del backend y frontend, y realizar pruebas generales del sistema para verificar su funcionamiento completo. Jimmy Feng y Alexander Montero presentaron el estado de la API, confirmando la estabilidad de los controladores, la consistencia de las respuestas y la protección por roles. José Barquero y Diego Salas revisaron la navegación general del frontend, incluyendo el manejo de sesiones, rutas protegidas y visualización de datos en tiempo real desde la API. Se ejecutaron pruebas

conjuntas en varias vistas: login, dashboard por rol, carga de documentos, visualización de notas y noticias. Se detectaron pequeños errores visuales y se asignaron correcciones rápidas para asegurar la presentación del proyecto. Además, se acordó documentar el despliegue del sistema en IIS, así como la estructura del repositorio para su presentación. Se definió el 21 de mayo como fecha límite para pruebas finales y cierre técnico. La reunión se realizó mediante Google Meet.

5.3 Actividades Realizadas por cada Estudiante

En esta parte del documento, se detallan las actividades realizadas por cada uno de los estudiantes en base al plan de trabajo, además de las referencias necesarias si alguna actividad consistió en investigación.

5.3.1 Jimmy Feng

- **25 de abril:** Configuró el entorno de trabajo en Azure DevOps y organizó el repositorio del backend.
- **26 de abril:** Estructuró el proyecto base en C# para el backend, incluyendo capas de modelos y contexto.
- **28 de abril:** Integró Swagger para documentación automática de la API.
- **30 de abril:** Desarrolló el CRUD completo para la entidad **Curso**.
- **5 de mayo:** Implementó el sistema de autenticación basado en MongoDB, incluyendo el controlador **AuthController**.
- **7 de mayo:** Protegió rutas del backend según el rol del usuario utilizando middlewares personalizados.
- **9 de mayo:** Desarrolló el CRUD para la entidad **Estudiante** conectada a MongoDB.
- **11 de mayo:** Implementó el CRUD de profesores con lógica de validación para evitar duplicados.
- **13 de mayo:** Trabajó en los endpoints de evaluación y su respectiva validación.
- **16 de mayo:** Desarrolló la funcionalidad CRUD para rubros de evaluación en la API.
- **18 de mayo:** Realizó la depuración de los endpoints y validó la estructura de las respuestas.

- **20 de mayo:** Ejecutó pruebas con Postman y ajustó detalles de serialización de datos [1].
- **21 de mayo:** Documentó toda la API utilizando anotaciones Swagger [2].
- **22 de mayo:** Participó en la integración final y revisión técnica del backend junto al equipo.

5.3.2 José Barquero

- **24 de abril:** Creó el repositorio del frontend y lo conectó con Azure DevOps.
- **25 de abril:** Colaboró en la organización de carpetas base del proyecto y configuraciones generales.
- **27 de abril:** Diseñó los estilos globales utilizando Bootstrap y configuraciones personalizadas [3].
- **29 de abril:** Implementó la pantalla de login para estudiantes con validación de formularios [4].
- **1 de mayo:** Desarrolló la vista del administrador para gestión de cursos.
- **3 de mayo:** Programó el formulario de carga de archivos Excel para la asignación de estudiantes.
- **8 de mayo:** Implementó la vista del profesor para la configuración de evaluaciones.
- **9 de mayo:** Diseñó el formulario para definir rubros de evaluación y lo conectó al backend.
- **13 de mayo:** Desarrolló la vista de noticias para estudiantes, consumiendo el API de publicaciones.
- **16 de mayo:** Implementó la validación de datos en la vista de notas del estudiante [5].
- **18 de mayo:** Encargado del ajuste final de estilos en componentes compartidos.
- **21 de mayo:** Realizó pruebas visuales y revisión de navegación para toda la aplicación frontend.
- **22 de mayo:** Participó en el cierre técnico e integración final del frontend con el backend.

5.3.3 Diego Salas

- **25 de abril:** Configuró la estructura inicial del proyecto Next.js para el frontend.
- **26 de abril:** Implementó la navegación inicial con React Router adaptado a Next.js [6].
- **28 de abril:** Desarrolló la pantalla de login para administrador.
- **30 de abril:** Implementó la redirección condicional por rol de usuario después del login.
- **4 de mayo:** Construyó la vista del administrador para la gestión de grupos académicos.
- **7 de mayo:** Diseñó la interfaz para la publicación de noticias por parte del profesor.
- **10 de mayo:** Implementó el formulario de entrega de tareas desde el panel del estudiante.
- **12 de mayo:** Programó la vista de entregas con validación visual y respuesta del backend.
- **14 de mayo:** Desarrolló la vista de notas del estudiante con paginación y ordenamiento.
- **19 de mayo:** Ejecutó pruebas de navegación entre vistas protegidas con roles.
- **21 de mayo:** Corrigió errores visuales menores y ajustó estilos en múltiples componentes.

5.3.4 Alexander Montero

- **27 de abril:** Configuró la conexión entre SQL Server y MongoDB en el backend.
- **4 de mayo:** Implementó la ruta para obtener grupos por curso y semestre en el controlador correspondiente.
- **6 de mayo:** Programó el middleware de sesión para proteger rutas sensibles según el rol del usuario.
- **10 de mayo:** Diseñó el modelo de profesor en MongoDB y su integración con el CRUD.

- **15 de mayo:** Agregó funcionalidad para generar reportes PDF desde el backend utilizando bibliotecas específicas [7].
- **17 de mayo:** Implementó relaciones explícitas entre curso y semestre usando Fluent API en EF Core [8].
- **26 de abril:** Configuró el enrutamiento centralizado y estructura de carpetas del frontend.
- **29 de abril:** Programó la pantalla de login para profesores y su redirección tras autenticación.
- **2 de mayo:** Construyó la vista para crear semestres como parte del módulo administrativo.
- **6 de mayo:** Desarrolló la vista del profesor para administración de documentos por curso.
- **11 de mayo:** Programó la interfaz del estudiante para visualizar carpetas de entrega.
- **17 de mayo:** Añadió funcionalidad para descargar reportes en PDF desde la vista del profesor.
- **20 de mayo:** Refactorizó rutas y lógica de navegación para mejorar la estructura del frontend.

5.3.5 Adrián Muñoz

- **24 de abril:** Configuró el repositorio inicial del backend y estructura base del proyecto.
- **25 de abril:** Participó en la configuración de Azure DevOps y organización de tareas por sprint.
- **29 de abril:** Desarrolló el modelo y lógica para la entidad **Curso**.
- **1 de mayo:** Implementó el CRUD básico para carreras en la API REST.
- **2 de mayo:** Programó el CRUD completo para la entidad **Semestre**, incluyendo validación del campo periodo [9].
- **8 de mayo:** Modeló la entidad **Estudiante** para MongoDB y configuró su estructura de colección.

- **12 de mayo:** Desarrolló la lógica para validar el acceso por rol en las rutas protegidas del backend.
- **14 de mayo:** Implementó la lógica para publicación de notas por parte del profesor.
- **17 de mayo:** Construyó la lógica para vincular evaluaciones con rubros y semestres.
- **19 de mayo:** Ejecutó pruebas generales con Postman para todos los controladores del backend.
- **20 de mayo:** Ajustó problemas de serialización JSON y formatos de respuesta para la API.
- **22 de mayo:** Participó en el cierre técnico y en la revisión cruzada entre módulos backend y frontend.

6 Conclusiones

La implementación de CEDigital ha permitido resolver de forma estructurada y centralizada el proceso de gestión académica mediante una plataforma híbrida que integra SQL Server, MongoDB y una interfaz web amigable. Esto proporciona trazabilidad y escalabilidad para futuras adaptaciones.

El uso de controladores RESTful para separar las operaciones sobre datos relacionales y documentos garantiza una arquitectura robusta, desacoplada y fácil de mantener tanto para estudiantes como para administradores del sistema.

Next.js demostró ser una herramienta eficiente para el desarrollo del frontend, permitiendo modularizar componentes, adaptarse fácilmente al diseño responsivo y mantener código limpio y reutilizable.

Las pruebas realizadas en un entorno real, utilizando IIS como servidor local, permitieron validar que el sistema puede integrarse sin fricciones en redes académicas institucionales bajo Windows.

El trabajo colaborativo, respaldado por una planificación detallada en Azure DevOps, facilitó el desarrollo progresivo de funcionalidades en ambos equipos (frontend y backend), lo que permitió alcanzar entregables funcionales por sprint y mantener trazabilidad del avance.

7 Recomendaciones

Para futuras mejoras, se sugiere la incorporación de autenticación con tokens JWT para aumentar la seguridad en las sesiones de usuario[10].

Es recomendable utilizar Docker para contenerizar tanto el backend como las bases de datos, facilitando el despliegue multiplataforma y pruebas en diferentes entornos[11].

El uso de pruebas automatizadas para cada controlador del backend puede mejorar la confiabilidad y facilitar futuras refactorizaciones[12].

La documentación del frontend puede enriquecerse con Storybook para visualizar y validar componentes de forma independiente[13].

Se recomienda implementar control de roles directamente en el frontend usando middlewares de Next.js para evitar condiciones de carrera en el acceso a rutas protegidas[14].

Referencias

- [1] Postman, *Postman Learning Center - API Documentation*, 2024. dirección: <https://learning.postman.com/docs/publishing-your-api/documenting-your-api/>.
- [2] Swagger, *Swagger - API Documentation Tools*, 2024. dirección: <https://swagger.io/docs/>.
- [3] B. Team, *Bootstrap v5.3 Documentation*, 2024. dirección: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.
- [4] R. H. Form, *React Hook Form - Simple React forms validation*, 2024. dirección: <https://react-hook-form.com/>.
- [5] Formik, *Formik + Yup - Form Validation in React*, 2024. dirección: <https://formik.org/docs/guides/validation>.
- [6] Vercel, *Routing: Pages and Navigation in Next.js*, Consultado en mayo de 2025, 2024. dirección: <https://nextjs.org/docs/routing/introduction>.
- [7] P. Project, *PDFsharp - A .NET library for processing PDF*, 2024. dirección: <https://pdfsharp.net/>.
- [8] M. Docs, *Entity Framework Core - Relationships*, 2024. dirección: <https://learn.microsoft.com/en-us/ef/core/modeling/relationships/>.
- [9] M. Docs, *Validation in Entity Framework Core*, 2024. dirección: <https://learn.microsoft.com/en-us/ef/core/modeling/entity-properties?tabs=data-annotations#validation>.
- [10] M. Jones, “Introduction to JSON Web Tokens,” 2023. dirección: <https://jwt.io/introduction>.
- [11] D. Inc., *What is Docker?* 2024. dirección: <https://www.docker.com/resources/what-container/>.
- [12] M. Docs, “Best Practices for Testing ASP.NET Core Web APIs,” 2024. dirección: <https://learn.microsoft.com/en-us/aspnet/core/test/overview?view=aspnetcore-7.0>.
- [13] S. Team, *Introduction to Storybook*, 2024. dirección: <https://storybook.js.org/docs/react/get-started/introduction>.
- [14] V. Inc., *Middleware – Next.js 13 Documentation*, 2025. dirección: <https://nextjs.org/docs/app/building-your-application/routing/middleware>.